

Supplementary Material for Continuous Deployment as Cost-Sensitive Decision-Making: When Contextual Bandits Outperform Static Rules and When They Don't

Abishek Kumar Giri
Stockton University

This document collects the supplementary material accompanying the main paper. It contains the per-claim validity classification (Appendix A), reproducibility instructions (Appendix B), extended threats to validity, and the apparatus and diagnostic tables (S1–S11) referenced from the main text. It is not self-contained and should be read alongside the main paper.

I. APPENDIX A: VALIDITY CLASSIFICATION

Labels: - **Demonstrated** — mechanism verified; result is robust across $n=30$ seeds, multiple configurations, and/or follows directly from code/math. Unlikely to reverse with more data. - **Suggestive** — result is real on the data collected, but scope is limited (synthetic, 2 projects, short trajectory). Directionally plausible; magnitude may shift. - **Speculative** — claim extrapolates beyond the experimental evidence, relies on analogy, or is a conditional prediction not yet tested.

A. 7.5 Drift Detection Conclusions Are Conditional on PageHinkley Calibration

Page-Hinkley at $\lambda_{\text{PH}} = 50$ fires 44 false alarms on 1,150 stationary steps in the ablation experiment, and 10.6 resets per 500-step trajectory in the drift-mode evaluation. The threshold was set by the PageHinkley default and never tuned to the cost stream's variance. All findings about drift adaptation performance — specifically that `linucb_with_drift_full` is 8–28% more expensive than plain LinUCB across all three drift modes — are conditional on this calibration. A lower λ_{PH} (fewer false alarms) or a different detector (ADWIN; Bifet & Gavaldà, 2007) may change the direction of the drift finding. We report the result for $\lambda_{\text{PH}} = 50$ as a measured limit of this detector on this cost stream; we make no claim about whether drift adaptation is intrinsically harmful.

B. 7.6 Thompson Sampling Propensities Are Intractable

The true selection probability for Thompson Sampling requires integrating over the posterior — intractable in closed form. We report propensity = 1.0 throughout. IPS correction cannot be applied to Thompson results even in principle. Thompson's cost estimates carry the standard replay bias plus this additional limitation.

C. 7.7 Seed Counts Below 30 Produce Misleading Direction Findings

Seed counts below approximately 30 produced misleading direction findings in our pilot runs ($n=5$ showed Thompson as the cheapest policy on synthetic data; $n=30$ reverses the direction with a non-overlapping CI). All headline numbers in this paper use ≥ 30 seeds and bootstrap seed 42.

D. 7.8 CI Outcome Is a Proxy for Deployment Failure, Not a Direct Measure

The entire evaluation chain — cost computation, policy update, and result reporting — treats the logged CI outcome (pass/fail) as a proxy for the deployment failure outcome a human engineer would observe in production. This proxy relationship is imperfect in both directions. A CI pass does not guarantee safe deployment: integration-test gaps, environment-specific faults, and configuration drift can produce production incidents on passing builds. A CI failure does not guarantee a production incident would have occurred: flaky tests, dependency version mismatches, and transient infrastructure errors produce false CI failures. The cost model therefore mis-prices a fraction of deployment decisions. The direction and magnitude of this mis-pricing depend on the test suite quality and build environment stability of each repository, neither of which is measured here. All cost claims should be read as costs under the CI-outcome proxy, not costs under the true deployment-failure outcome.

#	Claim	Section
1	The pending-reward buffer enforces the delayed-feedback invariant (no future information at decision time)	§2.2, F2
2	Removing the buffer improves cost by 1.1% on synthetic data	§5.3, F2
3	PageHinkley at $\lambda_{PH}=50$ fires 44 false alarms on 1,150 stationary steps	§5.3, F3
4	<code>linucb_with_drift_no_reset</code> is numerically identical to <code>linucb</code> in all three drift modes	§7.4, F12
5	Thompson Sampling produces nonzero seed variance ($\text{std}=54$ synthetic, $\text{std}=72$ real); LinUCB does not	§5.1, F4
6	Thompson’s 95% bootstrap CI [598, 648] does not overlap LinUCB=669.5; $p<0.0001$ paired bootstrap	§5.4, F9
7	Real GitHub Actions data contains 17–22 censored rewards per trajectory ($\approx 3\%$)	§5.4, F10
8	Cost weighting is the dominant ablation component: binary reward degrades cost by +31%	§5.3, F1; Abstract (+31%)
9	LinUCB over-blocks in the low-failure regime and costs 3.8% more than static rules	§5.4, F8; Abstract (3.8%)
10	Thompson Sampling outperforms LinUCB by 6.8% on real data (624 vs 669.5)	§5.4, F9; Abstract (6.8%)
11	Thompson and the static rule are statistically tied on real data (static=644.5 inside CI [598, 648])	§5.4, F9
12	Bandit advantage scales with failure cost: +19% at $2\times$ failure cost, +27% at lower block cost	§5.2, F5; Abstract (19%, 27%)
13	Cost-ratio sweep: LinUCB advantage monotone from -7% (5:1) to -49.6% (100:1)	§6.4, F11; Abstract (-7%, -49.6%)
14	Severe delay (120s) costs 1.0% more than default; short delay (30s) reduces cost by 1.5%	§5.2, F6
15	The aggregate bandit/static tie (1878 vs 1879) is a per-project cancellation artifact	§5.1, F7
16	Thompson’s posterior never fully collapses to the optimal arm (persistent canary allocation)	§1.3, §5.1
17	LinUCB exploration over static rules requires $\geq 40:1$ cost ratio in this synthetic setting	§6.4, operating envelope
18	Page-Hinkley drift resets at $\lambda=50$ are net-negative across all three drift modes	§6.4, F12
19	The 5.3% failure rate of psf/requests is a measured data property	§5.4; Abstract (5.3%)
20	The bandit framing helps when failure costs are high and context is informative	§1.2, Abstract
21	Per-project exploration calibration (adapting α to observed failure rate) would avoid over-blocking	§9
22	Longer trajectories would amortize reset overhead under drift	§6.4
23	The 20:1 default cost ratio sits at the operating envelope boundary where bandits break even	§1.4, Abstract

E. 7.9 Real-Data Evaluation Assumes Stationarity; No Drift Analysis Was Performed

The GitHub Actions evaluation treats each 300-step project trajectory as a stationary distribution and uses all steps for both learning and evaluation. If the true CI failure rate changes over the 300-step window — for example, due to a large dependency upgrade, a project refactor, or infrastructure migration — then the earlier steps of the trajectory correspond to a different distribution than the later steps. In that case, a policy that learns from early steps may be mis-calibrated for later steps, producing an underestimate of the cost gap between adaptive and non-adaptive policies. We performed no stationarity test on the real-data trajectories. The drift evaluation in §6.4 is conducted exclusively on synthetic data; its findings do not imply anything about whether the real GitHub Actions trajectories are stationary.

F. 7.10 Synthetic Failure Rates Are Unobservable Hidden State

The synthetic dataset assigns failure rates at the project level (smoke/ α : 15%, smoke/ β : 35%), but these rates are not part of the feature vector presented to the policy. From the policy’s perspective, the failure rate is a latent variable that must be inferred from the reward history. This creates a structural advantage for policies that converge quickly on the majority class: after seeing enough failures, LinUCB correctly assigns high probability to the block arm for smoke/ β , but the feature vector provides no direct signal that smoke/ β is a high-failure project. In a real setting, project metadata (team size, language, test coverage) could make the failure rate more directly observable. The synthetic evaluation therefore measures a lower bound on bandit performance: a policy with access to project-type features could converge faster.

The aggregate tie on the synthetic dataset does not reflect what would happen with a richer feature representation.

G. 7.11 Bootstrap Results Depend on a Single Fixed Seed; Different Seeds May Yield Different p-Values

All bootstrap confidence intervals and paired bootstrap p-values in this paper use seed 42 with 10,000 resamples (as noted in §2 and `experiments/run_robustness.py`). The bootstrap procedure is deterministic given this seed. We did not evaluate sensitivity of p-values or CI endpoints to the choice of bootstrap seed. For the F9 Thompson-vs-LinUCB result ($p<0.0001$, 0/10,000 bootstrap samples with Thompson mean $\geq$ LinUCB), the finding is robust: the p-value is at the floor for any 10,000-resample procedure and will not change under different seeds. For findings where the p-value is near a threshold — particularly the Thompson-vs-static “tied” result, where static=644.5 falls inside CI [598, 648] — a different bootstrap seed could shift the CI endpoints by a few units and change the coverage interpretation. Readers should treat the [598, 648] endpoints as approximate to ± 5 units under seed variation.

APPARATUS TABLES

- Table S1:** Context feature vector (13 dimensions).
- Table S2:** Policy specifications.
- Table S3:** Synthetic dataset statistics.
- Table S4:** Hyperparameters (synthetic experiment).
- Table S5:** Robustness conditions.
- Table S6:** Ablation variants.
- Table S7:** Real dataset statistics.

DIAGNOSTIC TABLES

- Table S8:** Per-project costs and optimal arm (synthetic).
- Table S9:** Delay-robustness sweep.
- Table S10:** Per-component ablation deltas.

Dimension	Feature	Normalisation
0	files_changed	$\div 50$
1	lines_added	$\div 1000$
2	lines_deleted	$\div 500$
3	src_churn	$\div 1500$
4	is_pr	binary
5	tests_run	$\div 200$
6	tests_added	$\div 10$
7	build_duration_s	$\div 180$
8	author_experience	$\div 10$
9	recent_failure_rate	$[0, 1]$
10	has_dependency_change	binary
11	has_risky_path_change	binary
12	bias	1.0

Policy	Description	Learning
static_rules	Deterministic threshold rule (files changed, failure rate, risky paths)	None
heuristic_score	Weighted risk score $\rightarrow$ thresholded action; fixed weights	None
linucb	Disjoint LinUCB (Li et al., 2010), $r = -cost$	UCB exploration
thompson	Bayesian linear TS (Agrawal & Goyal, 2013), $r = -cost$	Posterior sampling

Project	Builds	Failure rate	History span
smoke/alpha	600	15%	>365 days
smoke/beta	550	35%	>365 days

```
python -m experiments.run_ablations --seed $seed
done
```

Table S11: Per-project expected per-action costs (real data).

Results are written to `experiments/results/` (gitignored). The smoke dataset is at `data/raw/travistorrent_smoke.csv` (gitignored). All policy implementations, evaluation runners, and config files are tracked in version control.

II. APPENDIX B: EXPERIMENT REPRODUCIBILITY

```
# Main online replay (includes Thompson Sampling)
for seed in 0 1 2 3 4; do
    python -m experiments.run_bandits \
        --config experiments/configs/online_smoke.json \
        --seed $seed
done

# Real-data sanity check (requires data/raw/github_actions_real.csv)
for seed in 0 1 2 3 4; do
    python -m experiments.run_bandits \
        --config experiments/configs/real_github_actions.json \
        --seed $seed
done

# Robustness sweep
python -m experiments.run_robustness \
    --configs experiments/configs/online_smoke.json \
    experiments/configs/robustness_high_failure.json \
    experiments/configs/robustness_low_block.json \
    experiments/configs/robustness_short_delay.json \
    experiments/configs/robustness_long_delay.json \
    --seeds 0 1 2 3 4

# Ablation study
for seed in 0 1 2 3 4; do
```

Parameter	Value
α	1.0
λ	1.0
Delay model	$k_t = \max(1, \lceil \text{build_duration_s} / 60 \rceil)$
Cost matrix	Default (§2.3)
Seeds	0–4
flush_at_end	True

Condition	Key change
Default	—
High failure	deploy_failure: 10→20, canary_failure: 4→8
Low block	block_safe: 2→1, block_unknown: 2→1
Short delay	delay_step_seconds: 60→120
Long delay	delay_step_seconds: 60→30

Variant	What changes
no_drift (reference)	No drift reset
no_delay	Immediate updates (no buffer)
no_cost	Binary reward (-1/0 instead of -cost)
full	+ PageHinkley drift reset

Project	Runs	Failure rate	Span
psf/requests	300	5.3%	20 days
pallets/flask	300	23.3%	99 days

Project	Failure rate	<code>static_rules</code>	<code>linucb</code>	Optimal arm (expected cost)
<code>smoke/alpha</code>	15%	911.5	982.0	canary (1.45/step)
<code>smoke/beta</code>	35%	966.5	897.0	block (1.475/step)

Policy	Default (60s)	Short delay (120s)	Long delay (30s)
<code>static_rules</code>	1878	1878	1878
<code>linucb</code>	1879	1849	1897
<code>cost_sensitive_bandit</code>	1879	1849	1897

Component	Δ vs. <code>no_drift</code>	$\Delta\%$	Interpretation
Cost weighting $\rightarrow$ binary (<code>no_cost</code>)	+590	+31.4%	Destroys action–outcome asymmetry
Drift reset on stationary data (<code>full</code>)	+504	+26.8%	44 false alarms, each erasing learned weights
Delayed buffer $\rightarrow$ immediate (<code>no_delay</code>)	-21.5	-1.1%	Faster convergence; buffer costs temporal validity

Project	Failure rate	E[deploy]	E[canary]	E[block]	Optimal
<code>psf/requests</code>	5.3%	0.53	1.16	1.92	deploy
<code>pallets/flask</code>	23.3%	2.33	1.70	1.65	block